

The Brute → Better → Optimal Framework

Step 0: Problem Rephrasing & Understanding

✅ Always start here, not with coding.

- Read carefully, then **say the problem in your own words**.
- Identify inputs & outputs clearly.
- Clarify tricky terms (e.g., substring vs subsequence, path vs subpath, etc).

👉 Example:

“Find the length of the longest substring without repeating characters” → means I need a **continuous sequence** with all distinct chars, and return its length.

Step 1: Brute Force (Naive)

Ask yourself:

- “If I don’t care about time complexity, how would I solve it?”
- Usually → try **all possibilities**.

👉 Techniques:

- For strings/arrays → generate all subarrays/substrings.
- For graphs → DFS/BFS on every node.
- For optimization → try recursion.

✅ Why? Because brute force shows you:

- The **pattern of repeated work** (where optimization is possible).
 - The **time complexity bottleneck**.
-

Step 2: Bottleneck Detection

Now ask:

- “What’s making my brute force slow?”
- Is it recomputing the same thing many times?
- Is it checking too much repetitively?

👉 Common bottlenecks:

- Recomputing sums/unique checks → use hashing/prefix sums.
 - Traversing same states → use DP (store answers).
 - Looking up in arrays → use HashMap/Set.
-

Step 3: Optimization Intuition

Now think:

- “Can I reuse previous work?”
- “Can I avoid recomputation?”
- “Can I shrink the search space?”

👉 Typical strategies:

- **Sliding Window / Two Pointers** → when dealing with substrings/subarrays.
 - **Hashing** → fast lookups.
 - **Dynamic Programming** → overlapping subproblems.
 - **Greedy** → when local optimal → global optimal.
 - **Binary Search** → when problem has a monotonic property.
-

Step 4: Dry Run Small Example

Before coding, test your idea manually:

- Use a small input (length 5–6).
- Move through step-by-step.

- Confirm that your window/pointer/recurrence is working.

👉 This builds intuition + avoids bugs.

Step 5: Write & Refine Code

- Start with brute force → make sure it works.
 - Then optimize gradually (don't jump directly).
 - Add comments like "expand window", "shrink window" to remind logic.
-

Step 6: Analyze

- Time complexity (big-O).
 - Space complexity.
 - Edge cases (empty string, 1 char, all same char, very large input).
-

🏆 Example: Applying Framework to Longest Unique Substring

Step 0: Problem → Find max length substring with all distinct chars.

Step 1: Brute → Try all substrings, check distinct → $O(n^3)$.

Step 2: Bottleneck → Re-checking uniqueness again & again.

Step 3: Optimization → Use **sliding window** + HashSet to avoid re-checking.

Step 4: Dry run "geeksforgeeks".

Step 5: Code sliding window.

Step 6: Complexity → $O(n)$, space $O(k)$.

Mental Shortcut (Brute → Better → Optimal Flow)

Whenever you're stuck:

1. Can I **try all possibilities**? (brute force)
2. What's the **slow step**? (bottleneck)
3. Can I **reuse work / store info**? (optimize)
4. Which **technique fits best**? (sliding window, DP, greedy, etc)
5. Dry run → implement → analyze.

 This framework is universal — from **arrays to graphs to DP**.

Next time you face a problem, you'll have a **map** instead of just staring blankly.